

LESSON 1
**ABSTRACTION,
SORTING &
SEARCHING
ALGORITHMS**



**CCE 105L
DATA
STRUCTURES**



Java programmer



01

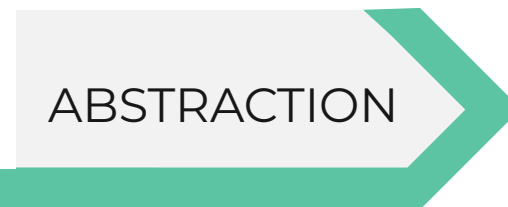
02

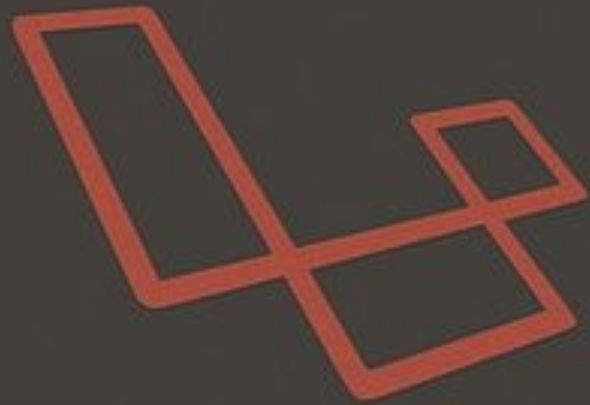
03

04

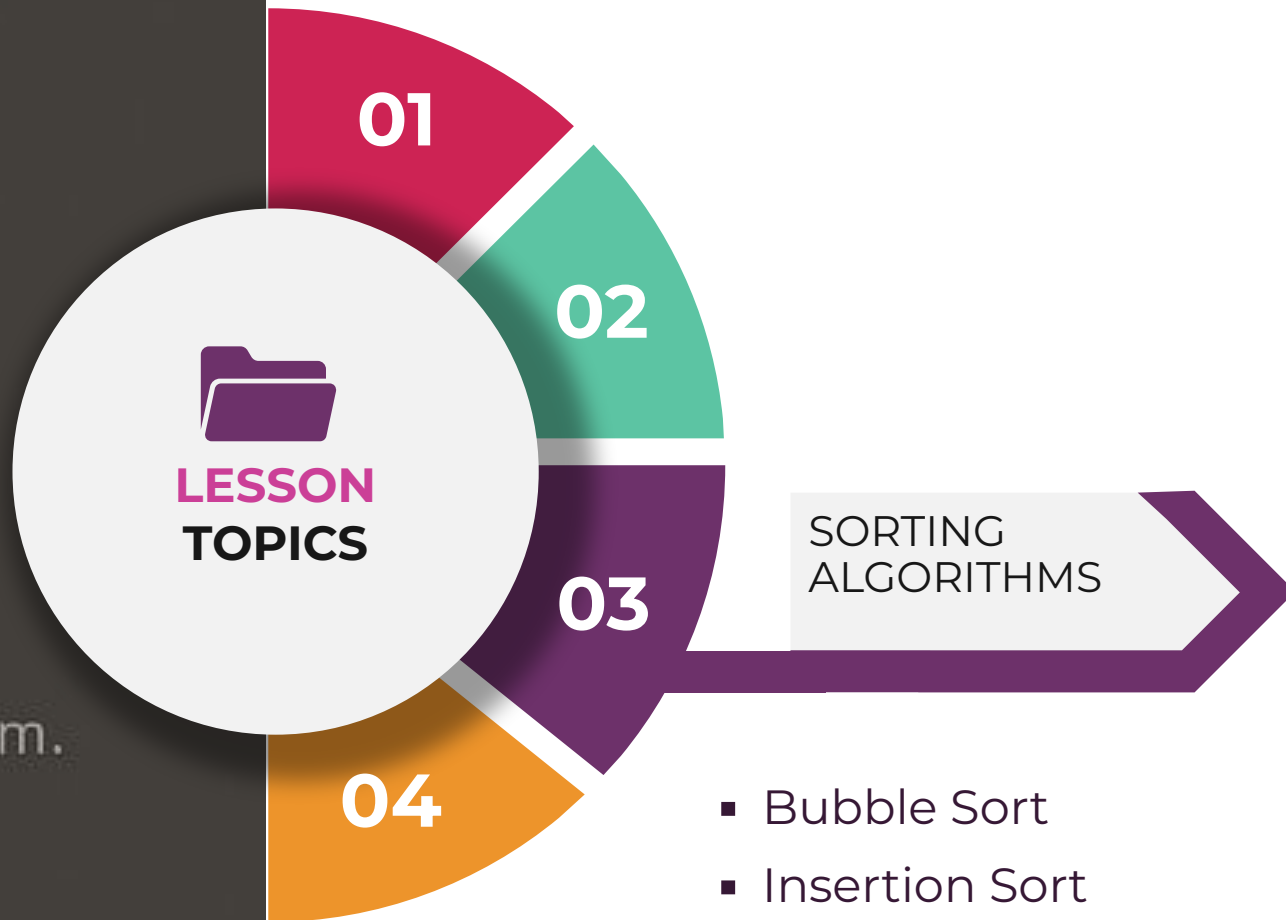
Problem Solving
Process

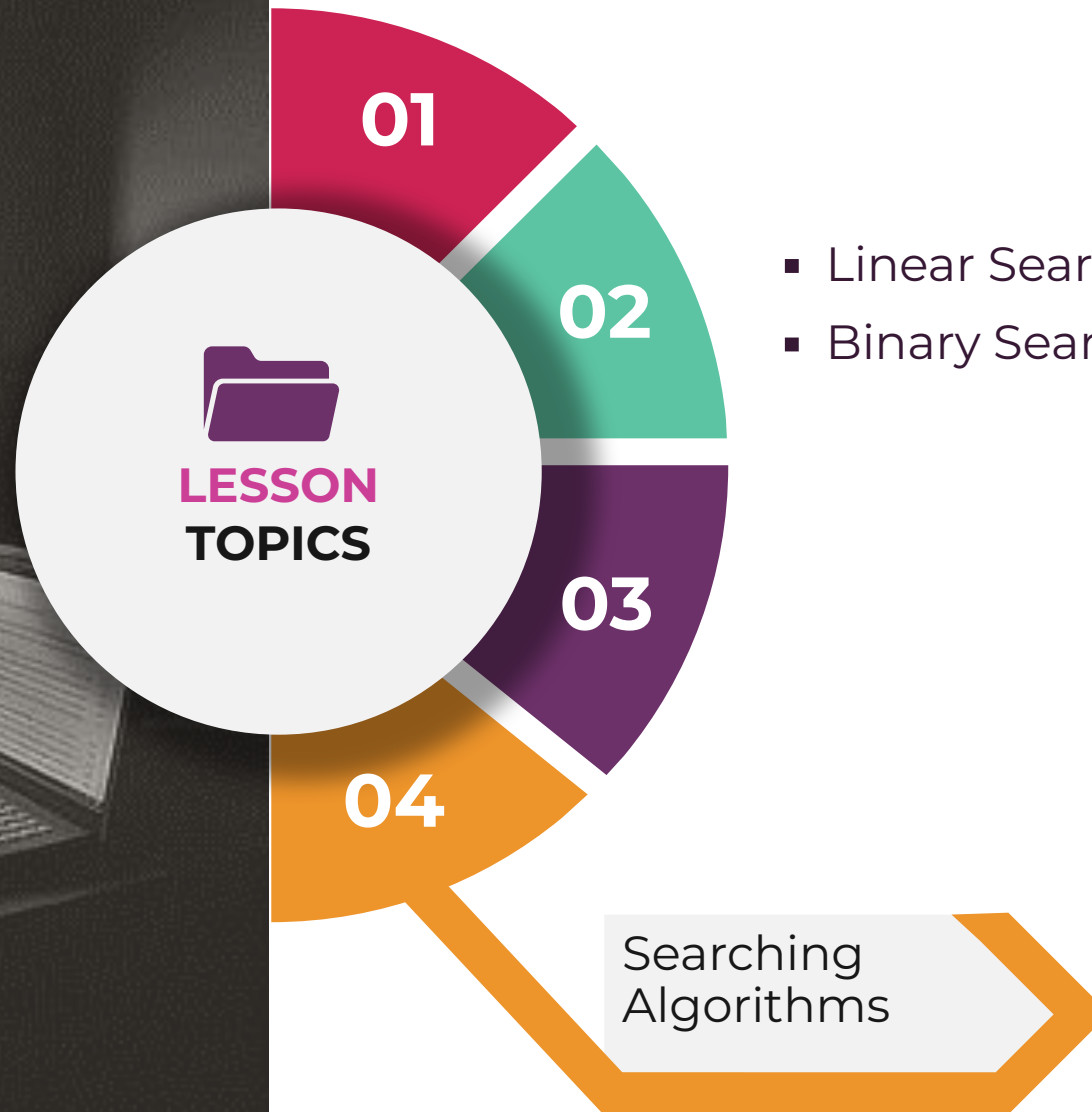






Programming is an art form.





DATA STRUCTURES AND ALGORITHM



Data Structure

is a arrangement of data in a computer's memory (or sometimes on a disk).

Data structures include arrays, linked lists, stack and binary trees, among others.

- A data structure is a specialized format for organizing, processing, retrieving and storing data.
- Data structures make it easy for users to access and work with the data they need in appropriate ways.

Algorithm

manipulates the data in these structures in various ways, such as searching for a particular data item and sorting the data.



Programming – is a problem-solving process.

Problem Solving Process

1

Problem domain

- input or the raw data to process
- output or the processed data

2

Machine domain

- Storage medium – consists of serially arranged bits that are addressable as a unit
- Processing unit – allows us to perform basic operations that include arithmetic, comparison, etc.

3

Solution domain

- links the problem and machine domains. It is at the solution domain where structuring of higher-level data structures and synthesis of algorithms are of concern.



Data Type

refers to the to the kind of data that variables can assume, hold or take on in a programming language and for which operations are automatically provided.

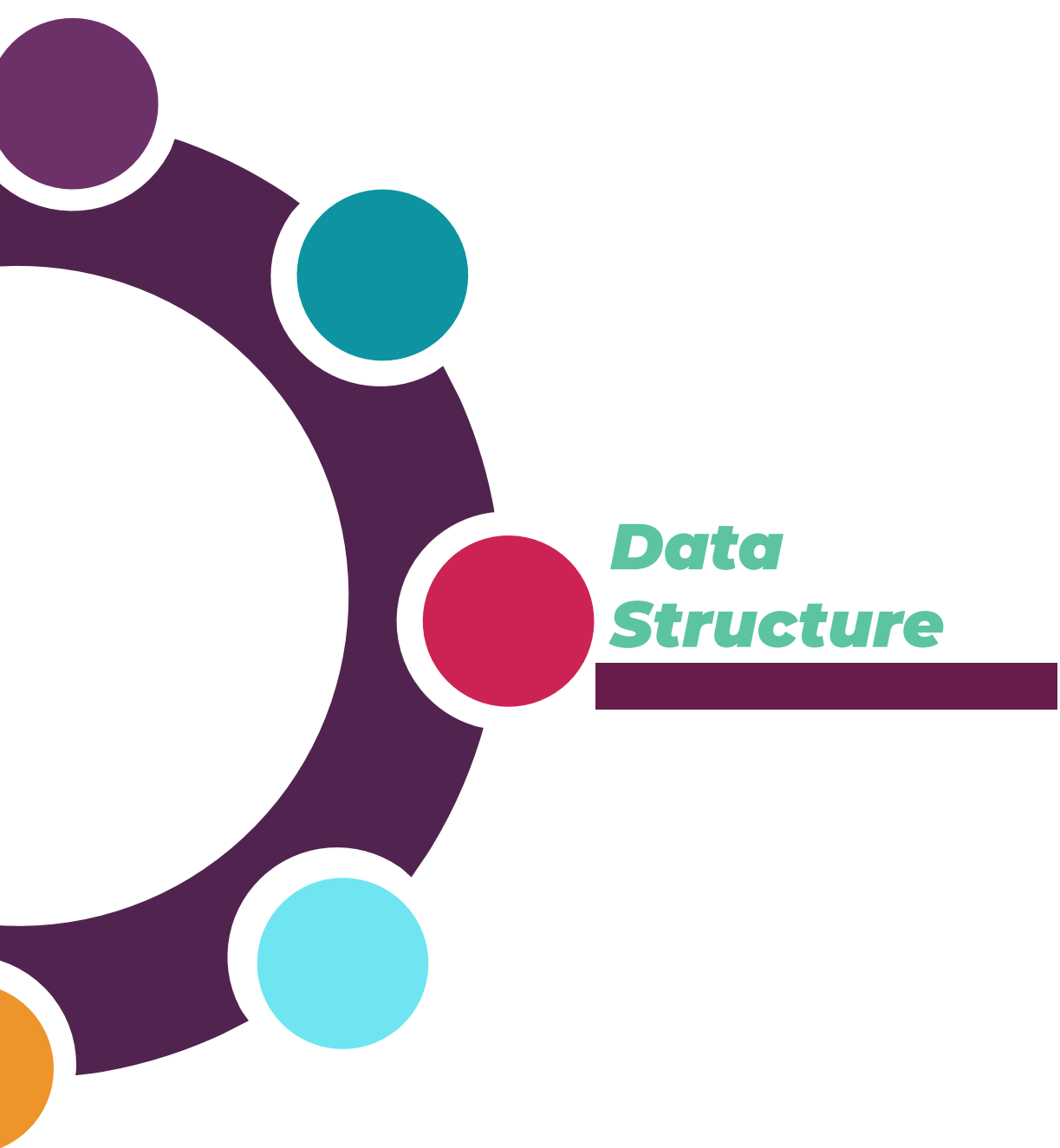
Keyword	Description
byte	Byte-length integer
short	Short integer
int	Integer
long	Long integer
float	Single-precision floating point
double	Double-precision floating point
char	A single character
boolean	A boolean value (true or false)



Abstract Data Type

is a mathematical model with a collection of operations defined on the model. It specifies the type of data stored. It specifies what its operations do but not how it is done. In Java, ADT can be expressed with an interface, which contains just a list methods.

- designed by composing data elements into a logical unit representing an abstract data type that has relevance to the algorithm or application
- An example of an abstract data type is a "customer name" that is composed of the character strings for "first name," "middle name" and "last name."



Data Structure

is the implementation of ADT in terms of the data types or other data structures. A data structure is modeled in Java by a class. Classes specify how operations are performed. In Java, to implement an ADT as a data structure, an interface is implemented by a class.



Algorithm

is a finite set of instructions which, if followed, will accomplish a task. It has five important properties: finiteness, definiteness, input, output and effectiveness.

Five Important Properties of Algorithm



1

Finiteness

means an algorithm must terminate after a finite number of steps.

2

Definiteness

is ensured if every step of an algorithm is precisely defined.

3

Input

is the domain of the algorithm which could be zero or more quantities.

4

Output

is the set of one or more resulting quantities which is also called the range of the algorithm.

5

Effectiveness

is ensured if all the operations in the algorithm are sufficiently basic that they can, in principle, be done exactly and in finite time by a person using paper and pen.



Example

```
public class Minimum {  
    public static void main(String[] args) {  
        int a[] = { 23, 45, 71, 12, 87, 66, 20, 33, 15, 69 };  
        int min = a[0];  
        for (int i = 1; i < a.length; i++) {  
            if (a[i] < min) min = a[i];  
        }  
        System.out.println("The minimum value  
            is: " + min);  
    }  
}
```



Mathematica I Functions

useful in the creation and analysis of algorithms.

1

Floor of x - the greatest integer less than or equal to x , where x is any real number.

Notation: $\lfloor x \rfloor$

Example:

$$\lfloor 3.14 \rfloor = 3 \quad \lfloor 1/2 \rfloor = 0 \quad \lfloor -1/2 \rfloor = -1$$

2

Ceiling of x - is the smallest integer greater than or equal to x , where x is any real number.

Notation: $\lceil x \rceil$

Example:

$$\lceil 3.14 \rceil = 4 \quad \lceil 1/2 \rceil = 1 \quad \lceil -1/2 \rceil = 0$$



Mathematica I Functions

useful in the creation and analysis of algorithms.



Modulo - Given any two real numbers x and y , $x \bmod y$ is defined as

" $-x \bmod n$ " is to find " $x \bmod n$ "

Example:

$$10 \bmod 3 = 1$$

$$24 \bmod 8 = 0$$

$$-5 \bmod 7 = -5$$

MATHEMATICAL FUNCTIONS



Example

$\lceil -3.14 \rceil$

Answer: -4

MATHEMATICAL FUNCTIONS



Example

$\lceil -3.14 \rceil$

Answer: -3

MATHEMATICAL FUNCTIONS



Example

$\lceil 5 \rceil$

Answer: 5

MATHEMATICAL FUNCTIONS



Example

$$\lceil -5 \rceil$$

Answer: -5

MATHEMATICAL FUNCTIONS



Example

$$\lceil -4.2 \rceil \bmod 3$$

Answer: -1

MATHEMATICAL FUNCTIONS



Example

$$\lfloor -4.2 \rfloor \bmod 3$$

Answer: -2



Example

$$\lfloor 4.2 \rfloor + \lceil 5.6 \rceil \bmod 2$$

Answer: 4

$$= 4 + 6 \bmod 2$$

$$= 4 + 0$$



Example

$$(\lfloor 4.2 \rfloor + \lceil -5.6 \rceil) + \lfloor 7.2 \rfloor \bmod 3$$

Answer: 0

$$\begin{aligned} &= (4 + -5) + 7 \bmod 3 \\ &= -1 + 1 \end{aligned}$$



Questions

MATHEMATICAL FUNCTIONS



Let's have a Quiz!

MATHEMATICAL FUNCTIONS



1. $\lfloor -5.3 \rfloor$
2. $\lfloor 6.14 \rfloor$
3. $\lceil -4.5 \rceil$
4. $8 \bmod 4$
5. $3 \bmod -4$
6. $\lceil 4.3 \rceil$
7. $-5 \bmod 2$
8. $10 \bmod 11$
9. $(15 \bmod -9) + \lfloor 4.3 \rfloor$
10. $\lceil 7.9 \rceil$

11. $(\lfloor 4.3 \rfloor + \lceil -3.7 \rceil) + (15 \bmod 3) + \lfloor -2.1 \rfloor$
12. $20 \bmod \lfloor -4 \rfloor$
13. $\lceil 12.4 \rceil \bmod \lfloor -3.2 \rfloor$
14. $\lfloor 5.6 \rfloor \bmod \lceil -4.1 \rceil + ((\lfloor 3.1 \rfloor + \lceil -5.2 \rceil) \bmod -4)$
15. $\lfloor 4.2 \rfloor + \lceil 5.6 \rceil \bmod 2$
16. $\lfloor -3.14 \rfloor$

MATHEMATICAL FUNCTIONS



Let us answer your quiz!

MATHEMATICAL FUNCTIONS



1.	$\lfloor -5.3 \rfloor$	-6
2.	$\lfloor 6.14 \rfloor$	6
3.	$\lceil -4.5 \rceil$	-4
4.	$8 \bmod 4$	0
5.	$3 \bmod -4$	3
6.	$\lceil 4.3 \rceil$	5
7.	$-5 \bmod 2$	-1
8.	$10 \bmod 11$	10
9.	$(15 \bmod -9) + \lfloor 4.3 \rfloor$	10
10.	$\lceil 7.9 \rceil$	8

MATHEMATICAL FUNCTIONS



$$11. \quad (\lfloor 4.3 \rfloor + \lceil -3.7 \rceil) + (15 \bmod 3) + \lfloor -2.1 \rfloor$$

$$(4 + -3) + 0 + -3 = -2$$

$$12. \quad 20 \bmod \lfloor -4 \rfloor$$

$$20 \bmod -4 = 0$$

$$13. \quad \lceil 12.4 \rceil \bmod \lfloor -3.2 \rfloor$$

$$13 \bmod -4 = 1$$

$$14. \quad \lfloor 5.6 \rfloor \bmod \lceil -4.1 \rceil + ((\lfloor 3.1 \rfloor + \lceil -5.2 \rceil) \bmod -4)$$

$$= 5 \bmod -4 + ((3 + -5) \bmod -4)$$

$$= 1 + (-2 \bmod -4)$$

$$= 1 - 2$$

$$= -1$$

$$15. \quad \lfloor 4.2 \rfloor + \lceil 5.6 \rceil \bmod 2$$

$$4 + 6 \bmod 2 = 4$$

$$16. \quad \lfloor -3.14 \rfloor$$

$$-4$$

SORTING ALGORITHMS



A **Sorting Algorithm** is used to rearrange a given array or list elements according to a comparison operator on the elements. The comparison operator is used to decide the new order of element in the respective data structure.

- 1 Bubble Sort
- 2 Insertion Sort
- 3 Selection Sort



Bubble Sort

- is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in wrong order.

- is also known as **exchange sort**.

In bubble sort algorithm array is traversed from 0 to the length-1 index of the array and compared one element to the next element and swap values in between if the next element is less than the previous element.

In other words, bubble sorting algorithm compare two values and put the largest value at largest index. The algorithm follow the same steps repeatedly until the values of array is sorted.

Bubble Sort

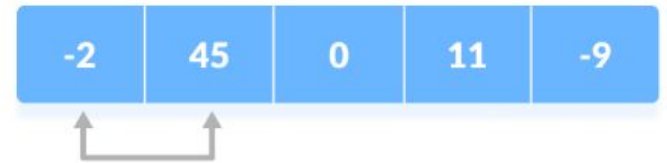
Starting from the first index, compare the first and the second elements. If the first element is greater than the second element, they are swapped.

Now, compare the second and the third elements. Swap them if they are not in order.

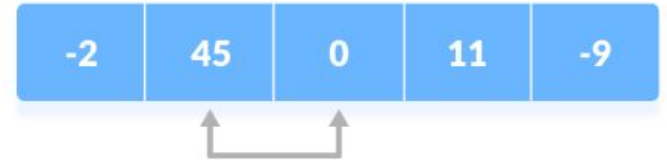
The above process goes on until the last element.

step = 0

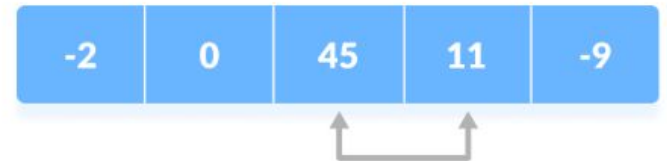
i = 0



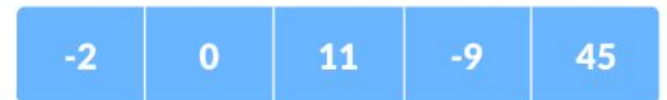
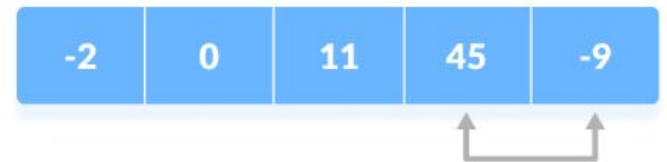
i = 1



i = 2



i = 3



Bubble Sort

The same process goes on for the remaining iterations. After each iteration, the largest element among the unsorted elements is placed at the end.

In each iteration, the comparison takes place up to the last unsorted element.

The array is sorted when all the unsorted elements are placed at their correct positions.

step = 1

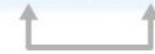
i = 0

-2	0	11	-9	45
----	---	----	----	----



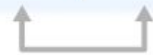
i = 1

-2	0	11	-9	45
----	---	----	----	----



i = 2

-2	0	11	-9	45
----	---	----	----	----



-2	0	-9	11	45
----	---	----	----	----

Bubble Sort

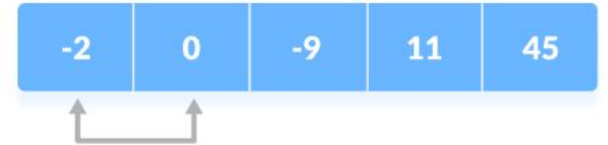
The same process goes on for the remaining iterations. After each iteration, the largest element among the unsorted elements is placed at the end.

In each iteration, the comparison takes place up to the last unsorted element.

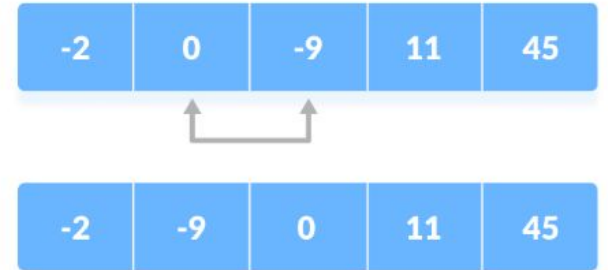
The array is sorted when all the unsorted elements are placed at their correct positions.

step = 2

i = 0

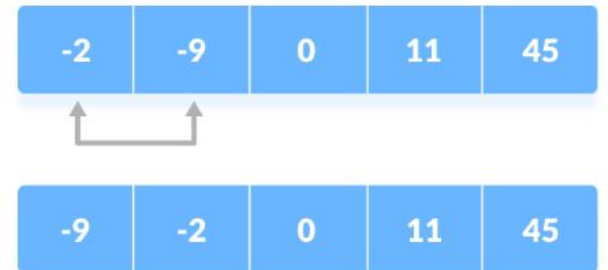


i = 1



step = 3

i = 0





Bubble Sort

```
import java.util.Arrays;
class BubbleSort {
    void bubbleSort(int array[]) {
        int size = array.length;
        for (int i = 0; i < size - 1; i++)
            for (int j = 0; j < size - i - 1; j++)
                if (array[j] > array[j + 1]) {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                }
    }
    public static void main(String args[]) {
        int[] data = { -2, 45, 0, 11, -9 };
        BubbleSort bs = new BubbleSort();
        bs.bubbleSort(data);
        System.out.println("Sorted Array in Ascending Order:");
        System.out.println(Arrays.toString(data));
    }
}
```



Sample
Program



Insertion Sort

- is a simple sorting algorithm that works the way we sort playing cards in our hands.

We assume that the first card is already sorted then, we select an unsorted card. If the unsorted card is greater than the card in hand, it is placed on the right otherwise, to the left. In the same way, other unsorted cards are taken and put at their right place.

A similar approach is used by insertion sort.

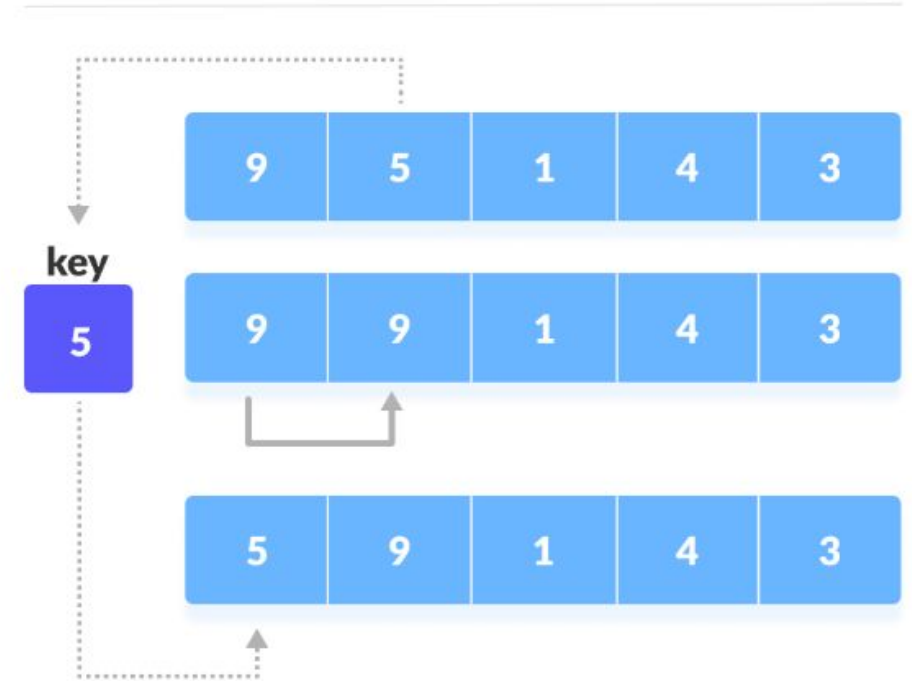
Insertion sort is a sorting algorithm that places an unsorted element at its suitable place in each iteration.

Insertion Sort

1. The first element in the array is assumed to be sorted. Take the second element and store it separately in key.

Compare key with the first element. If the first element is greater than key, then key is placed in front of the first element.

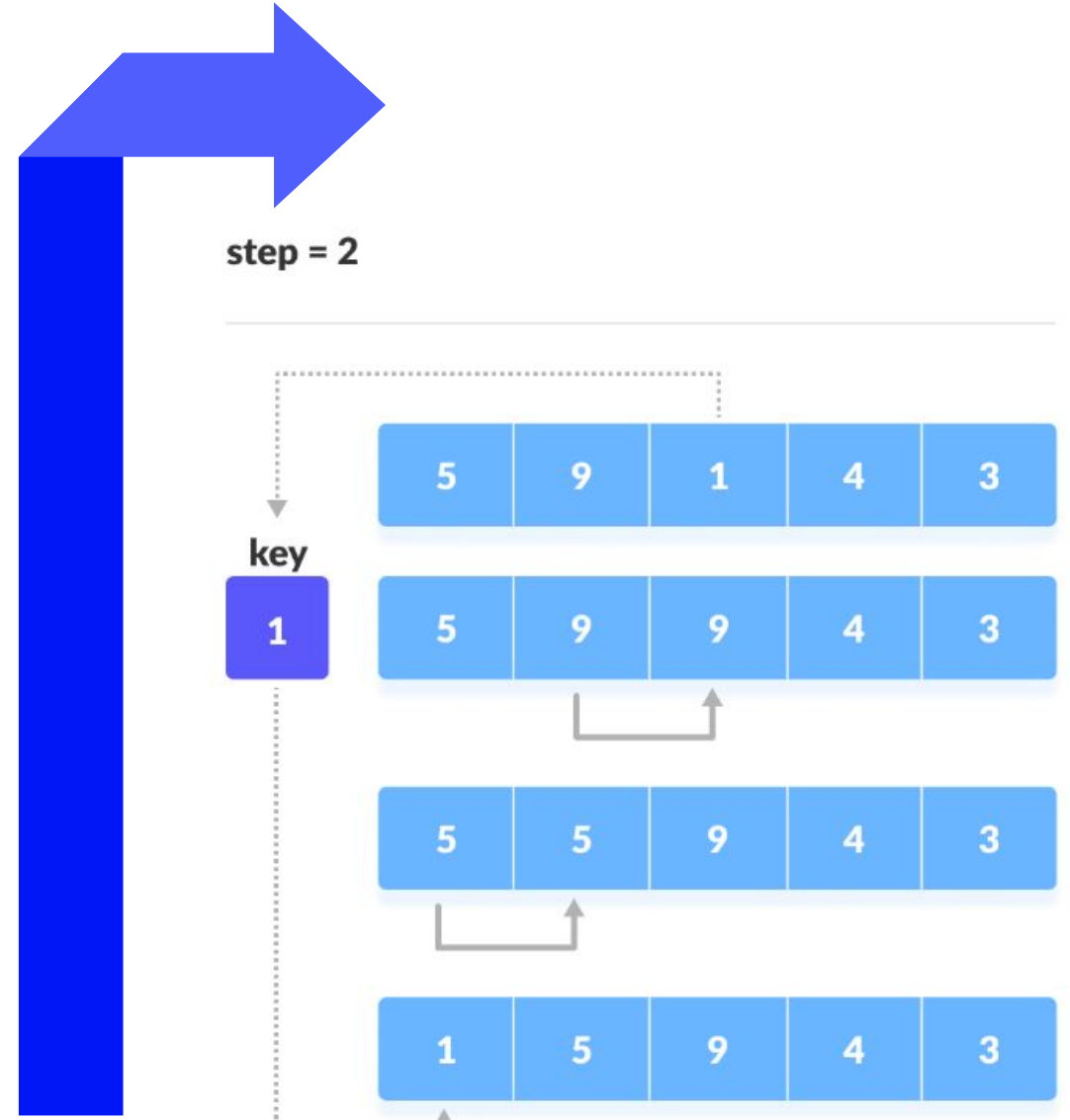
step = 1



Insertion Sort

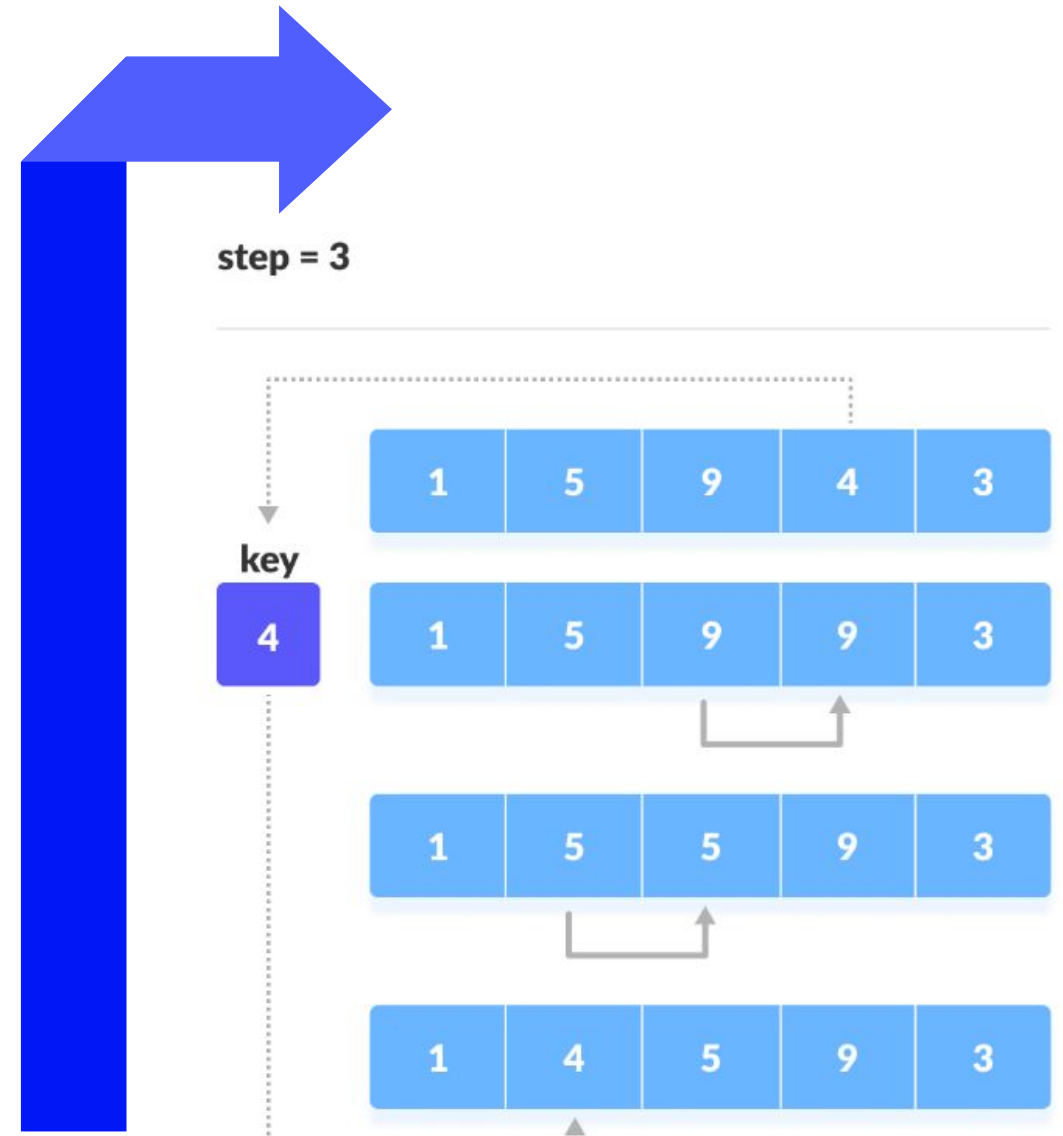
2. Now, the first two elements are sorted.

Take the third element and compare it with the elements on the left of it. Place it just behind the element smaller than it. If there is no element smaller than it, then place it at the beginning of the array.



Insertion Sort

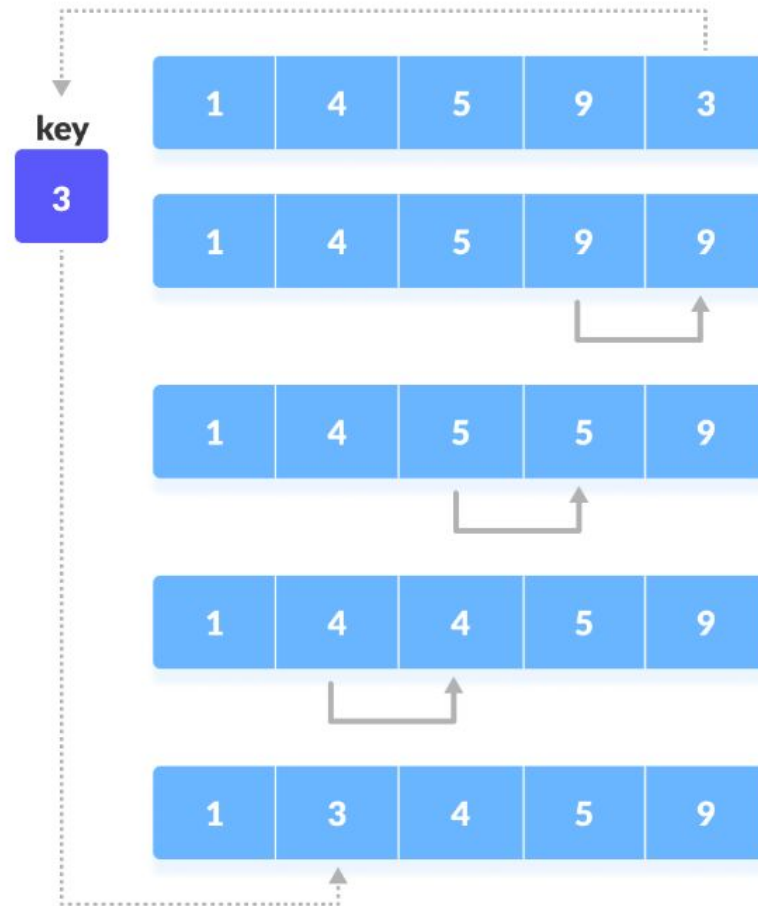
3. In a similar way, place every unsorted element at its correct position.



Insertion Sort

3. In a similar way, place every unsorted element at its correct position.

step = 4



Insertion Sort

```
import java.util.Arrays;
class InsertionSort {
    void insertionSort(int array[]) {
        int size = array.length;
        for (int step = 1; step < size; step++) {
            int key = array[step];
            int j = step - 1;
            while (j >= 0 && key < array[j]) {
                array[j + 1] = array[j];
                --j;
            }
            array[j + 1] = key;
        }
    }
    public static void main(String args[]) {
        int[] data = { 9, 5, 1, 4, 3 };
        InsertionSort is = new InsertionSort();
        is.insertionSort(data);
        System.out.println("Sorted Array in Ascending Order: ");
        System.out.println(Arrays.toString(data));
    }
}
```

Sample Program



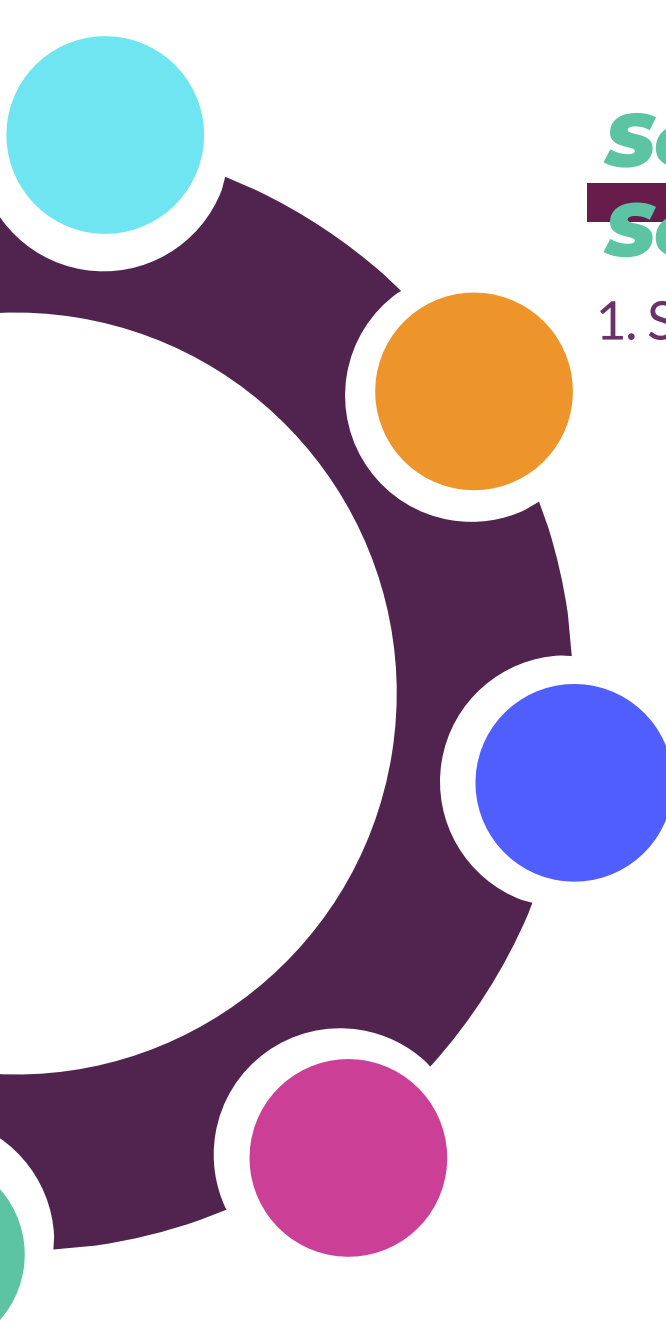
Selection Sort

- is an algorithm that selects the smallest element from an unsorted list in each iteration and places that element at the beginning of the unsorted list.

The selection sort algorithm sorts an array by repeatedly finding the minimum element (considering ascending order) from unsorted part and putting it at the beginning. The algorithm maintains two subarrays in a given array.

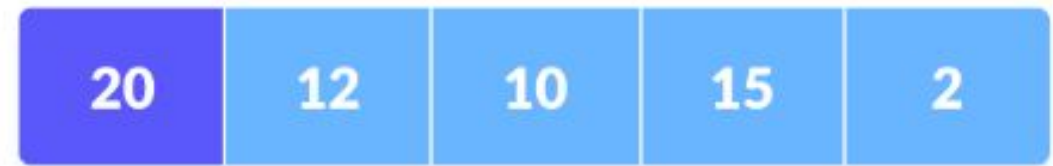
- 1) The subarray which is already sorted.
- 2) Remaining subarray which is unsorted.

In every iteration of selection sort, the minimum element (considering ascending order) from the unsorted subarray is picked and moved to the sorted subarray.



Selection Sort

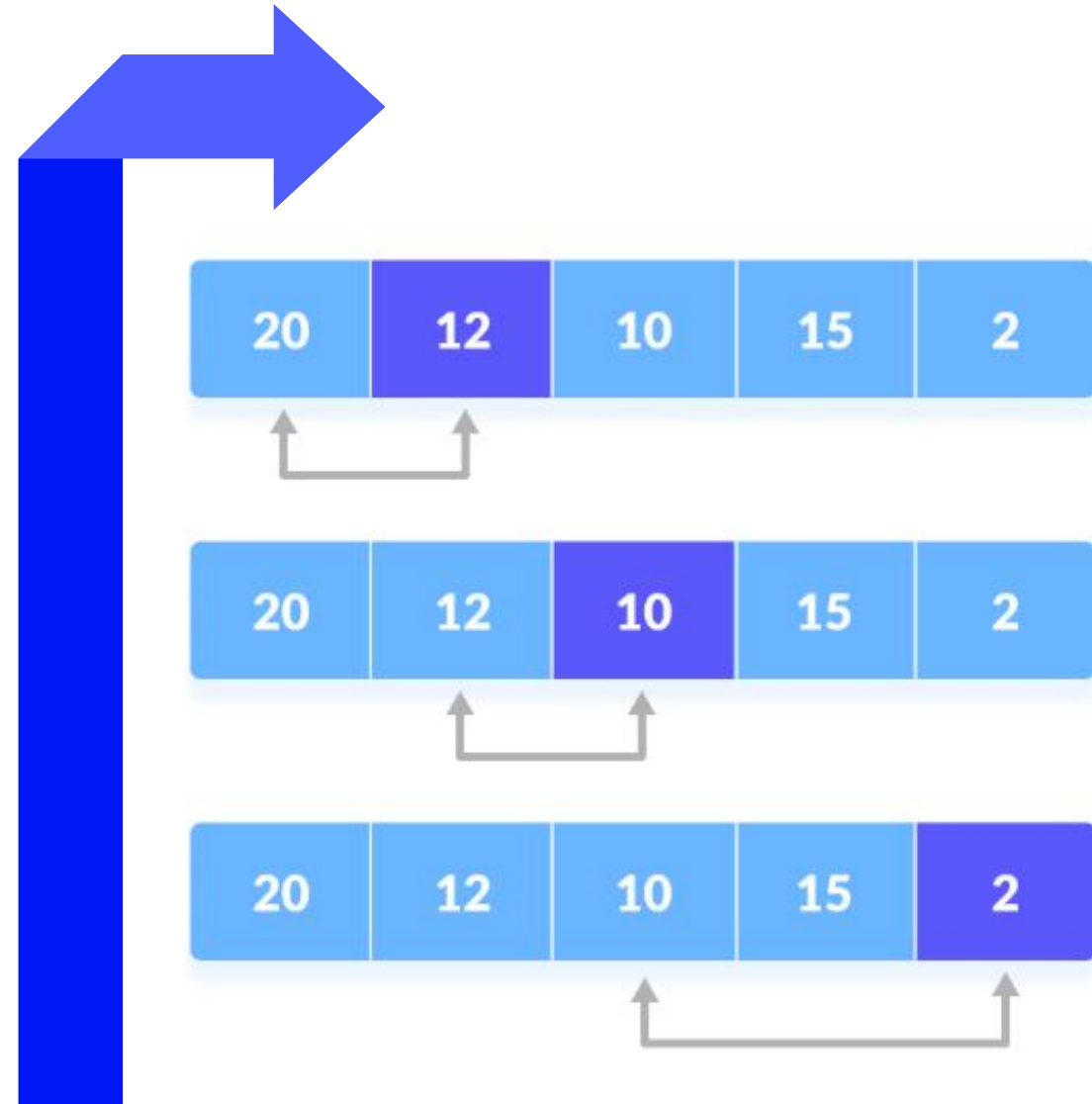
1. Set the first element as minimum.



Selection Sort

2. Compare minimum with the second element. If the second element is smaller than minimum, assign second element as minimum.

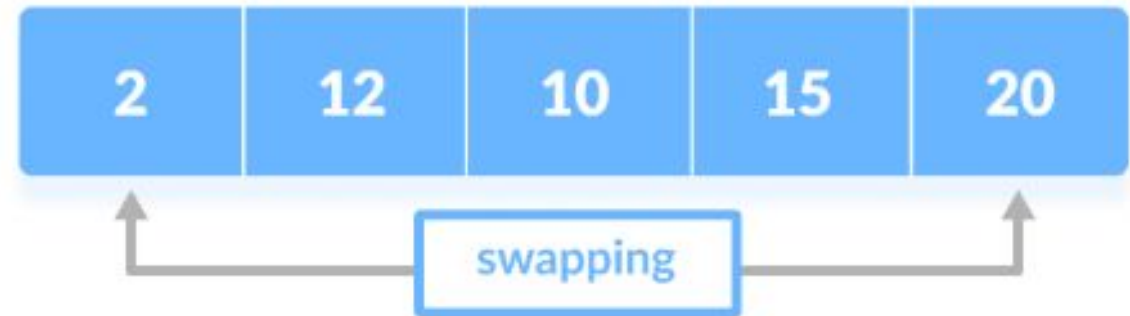
Compare minimum with the third element. Again, if the third element is smaller, then assign minimum to the third element otherwise do nothing. The process goes on until the last element.





Selection Sort

3. After each iteration, minimum is placed in the front of the unsorted list.

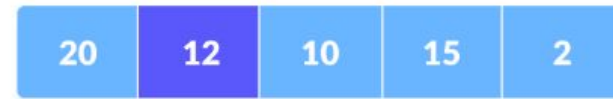


Selection Sort

4. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.

step = 0

i = 0



min value
at index 1

i = 1



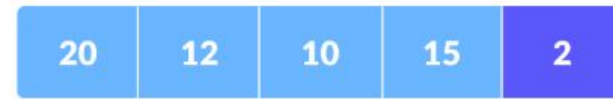
min value
at index 2

i = 2

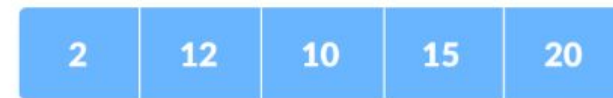


min value
at index 2

i = 3



min value
at index 4



swapping

Selection Sort

4. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.

step = 1

i = 0



min value
at index 2

i = 1



min value
at index 2

i = 2



min value
at index 2



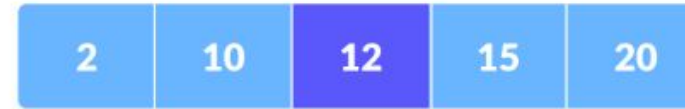
swapping

Selection Sort

4. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.

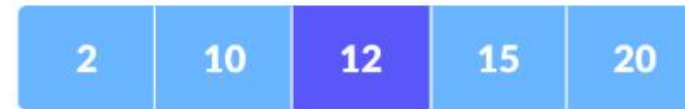
step = 2

i = 0

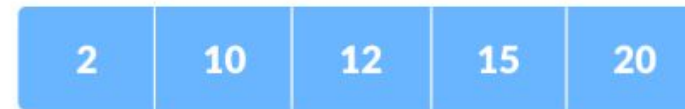


min value
at index 2

i = 2



min value
at index 2



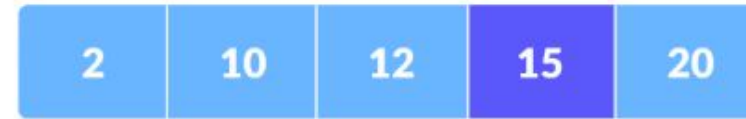
already in place

Selection Sort

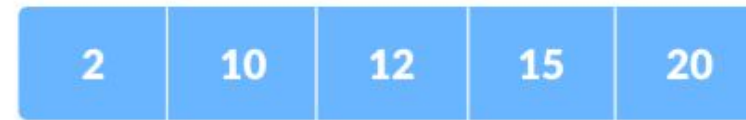
4. For each iteration, indexing starts from the first unsorted element. Step 1 to 3 are repeated until all the elements are placed at their correct positions.

step = 3

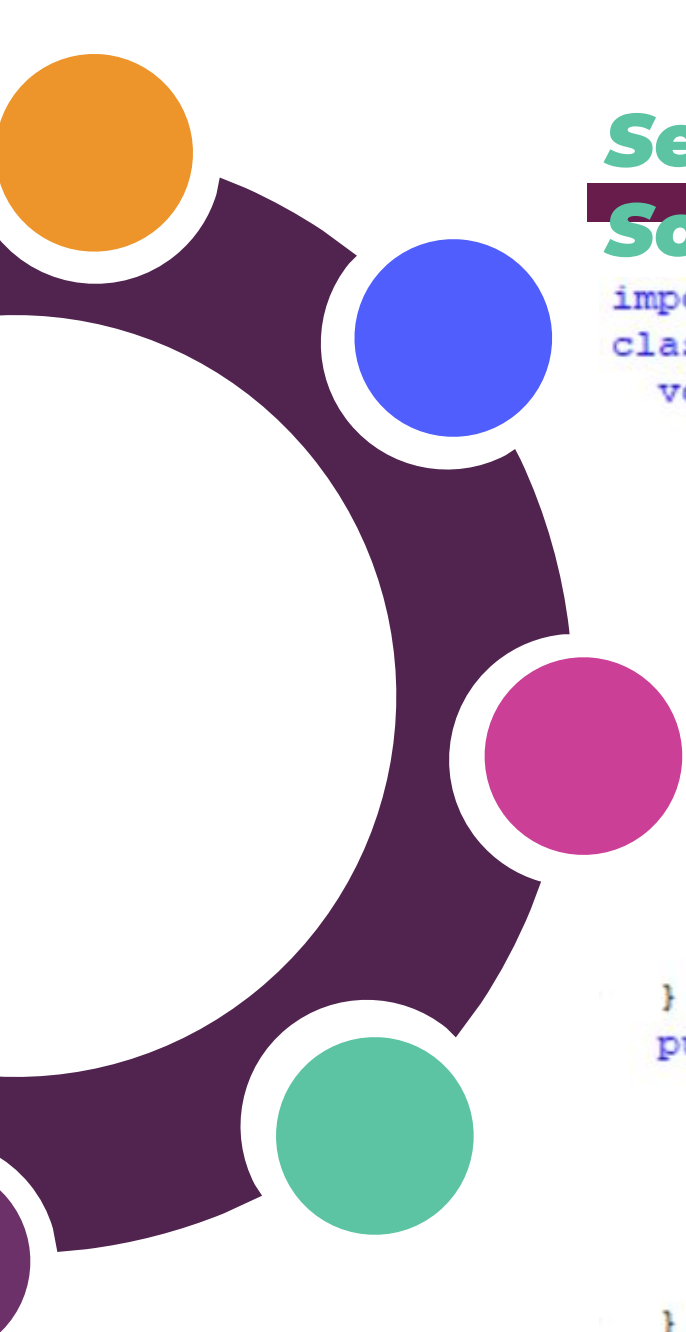
i = 0



min value
at index 3



already in place



Selection Sort

```
import java.util.Arrays;
class SelectionSort {
    void selectionSort(int array[]) {
        int size = array.length;
        for (int step = 0; step < size - 1; step++) {
            int min_idx = step;
            for (int i = step + 1; i < size; i++) {
                if (array[i] < array[min_idx]) {
                    min_idx = i;
                }
            }
            int temp = array[step];
            array[step] = array[min_idx];
            array[min_idx] = temp;
        }
    }
    public static void main(String args[]) {
        int[] data = { 20, 12, 10, 15, 2 };
        SelectionSort ss = new SelectionSort();
        ss.selectionSort(data);
        System.out.println("Sorted Array in Ascending Order: ");
        System.out.println(Arrays.toString(data));
    }
}
```



Sample Program



Thanks!
Questions?